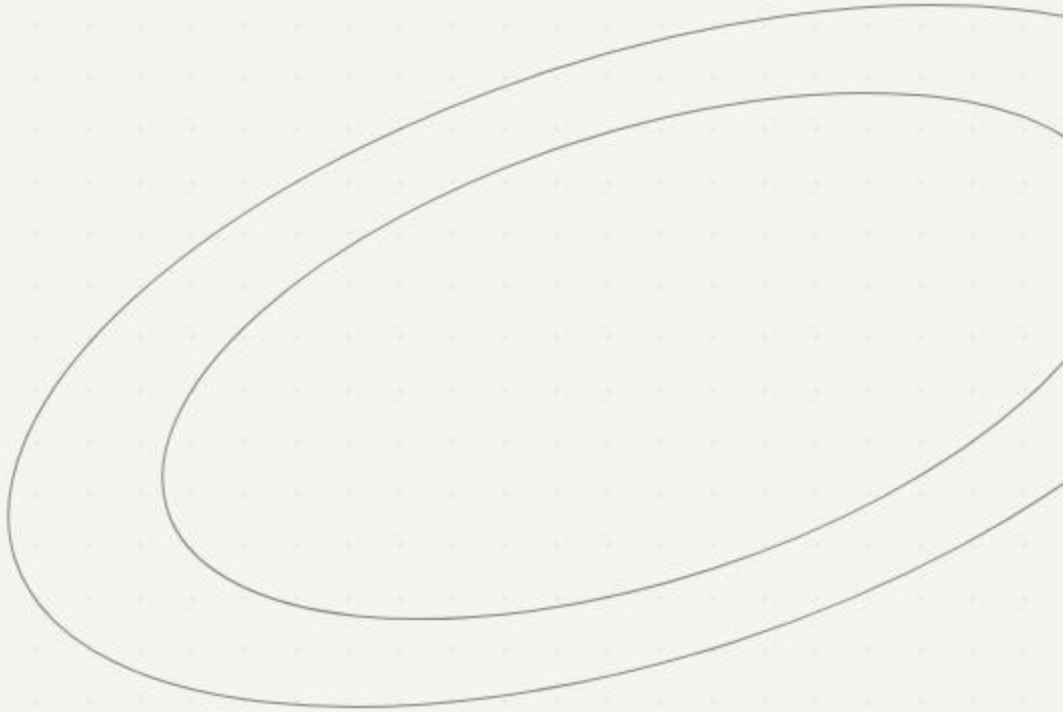
天体力学 × AI · 2026

从下一个 token 到下一个定理

一个模型如何工作，以及我如何使用它



刘博文

From Next Token to Next Theorem

全景：从 token 到定理

点击章节卡片跳转；点击小标签直达关键页。

00 · 开场与 TOKEN

Token

十个整数 · 词表 · 计费

p. 3–8

10 个整数 · 词表

01 · 模型

一个函数

概率分布 · 无状态

p. 9–10

概率分布 · 无状态

02 · 训练与问题求解

从奖励中学习

RL 循环 · 数学与代码 · GRPO

p. 11–23

RL 循环 · GRPO 公式 · REFLEX

03 · 系统与工程

把模型放进系统

上下文 · Agent · 代码审查

p. 24–40

Claude Code · 系统总图

04 · AI × 数学

从工具到搜索

加速 · 时间线 · 共同模式

p. 41–46

时间线 · 共同模式

05 · 结论

回到我的实践

工作方式 · 复利 · 收束

p. 47–53

两次尝试 · 结尾

6 个章节 · 52 页 · 点击任意卡片开始；`N` 显示讲稿，`F` 全屏。

模型看到的，是 10 个整数

EN · 一句话

Celestial mechanics is the poetry of orbits.

Cel	estial	_mechanics	_is	_the	_poetry	_of	_or	bits	.
69126	90010	52347	382	290	38153	328	503	24043	13

46 个字符 → 10 个 token → 10 个整数 · _ 表示前导空格

中文 · 《天体力学基础》内容简介

本书系统地介绍了天体力学

本	书	系统	地	介绍	了	天	体	力	学
4087	20121	31936	5557	37887	4531	867	7074	11343	8590

12 个字符 → 10 个 token → 10 个整数 · 系统 / 介绍 各占一个 token

01

这串整数是怎么来的?

02

它怎么预测下一个?

03

我们怎么用它干活?

切分是查表的结果

三体问题 is a classic problem in celestial mechanics.

三	体	问题	_is	_a	_classic	_problem	_in	_celestial	_mechanics	.
3236	7074	28917	382	261	13686	4792	306	134437	52347	13

_ 表示前导空格 · Celestial → Cel + estial

≈5
字符 / 英文单词
含空格，常见值

≈4
字符 / token
英文经验值

≈1.3
token / 英文单词
≈ 5 ÷ 4

5–20 万
常见词表大小
50k – 200k

为什么 字母级：信息少，序列长 4–5 倍 → 词级：词表爆炸（百万级） → **子词：折中**

GPT-2 50k · GPT-4 100k · Llama 3 128k · GPT-4o 200k

> 子词不是语义单位，是**序列长度**与**词表大小**之间的工程折中。

英文：1 token ≈ 4 字符 ≈ 0.75 词

词表：一张固定的映射表

《天体力学基础》 · 行号 = ID · 按 ID 排序

ID	片段	
867	天	
7074	体	
8590	学	
11343	力	
20121	书	
31936	系统	
78580	基础	← 本次查询
199999	<|endoftext|>	特殊

只展示 8 行 共 200,019 行

词表大小 = 行数

200,019

o200k · GPT-4o 代

- **0**
- **50k** GPT-2
- **100k** GPT-4 代
- **128k** Llama 3
- **200k** GPT-4o 代

200,019 × 4,096 ≈ **8.2 亿参数**

> ID 是**行号**；换模型，就是换一张表。

Token 是模型的单位，也是账单的单位



上月用量 · 平均每次 ≈ 110k token · ≈ 1 分钱

≈ 0.3 token

1 个英文字符

≈ 0.6 token

1 个中文字符

直观理解：一个词、一个数字、一个符号 ≈ 1 token；不同 tokenizer 不同，以返回的 usage 为准。

> Token 既是模型的最小单位，也是账单的最小单位。

一次调用，钱花在哪

价格[2]	百万tokens输入（缓存命中）	空闲时段	0.02元
		高峰时段	0.04元
	百万tokens输入（缓存未命中）	空闲时段	1元
		高峰时段	2元
	百万tokens输出	空闲时段	4元
		高峰时段	8元

deepseek-flash · 1M 上下文 · 384K 输出 · 元 / 百万 token

1M 上下文 ≈ 一部《三体》三部曲

一次调用 · 空闲时段

输入 100k + 输出 2k

¥0.01 缓存命中

¥0.11 缓存未命中

> 同一段上下文，命中缓存与不命中，差一个**数量级**。

文字先变成 ID，再变成向量

片段

问题

切分后的单位

→

ID

28917

词表第 28917 行

→

向量

0.12 −0.44 0.08 …

4096 维 · 训练得到

> ID 是**地址**，向量是**内容**。

这一步叫 embedding：每一维没有名字，但距离有意义。

模型的工作：输入 → 概率分布

输入 token

从 | 下一个 | token | 到 | 下一个

每个 token 已经是一个向量

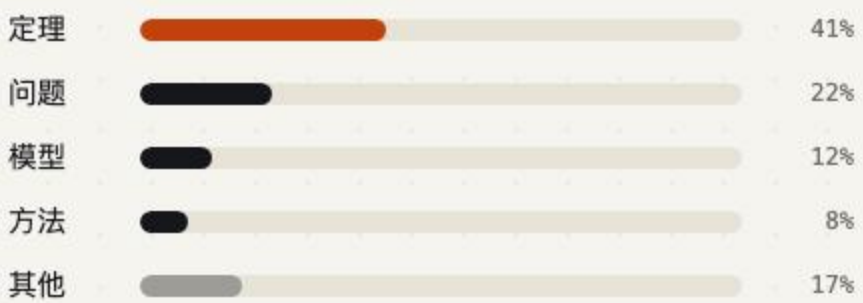
模型

Transformer

N 层 · 自注意力

GPT · Llama · Qwen 都是变体

softmax(z)



$$z = f_\theta(x_{1:t}) \in \mathbb{R}^{|V|} \qquad p_\theta(y \mid x_{1:t}) = \frac{\exp(z_y)}{\sum_{v \in V} \exp(z_v)}$$

$|V| = 200{,}019$

$\Sigma_y\, p_\theta(y \mid x_{1:t}) = 1$

> 它不查数据库，也不“理解”；它给词表里每个 token **打分**，再把分数变成**概率**。

模型没有记忆

调用 A

system	你是天体力学助手
user	希尔区是什么?
assistant	希尔区是……
user	它为什么重要?
assistant	因为它界定了……

调用 B · 新的一次

system	你是天体力学助手
user	它为什么重要?
assistant	（不知道“它”指什么）

> 模型无状态；所谓“对话”，是客户端把历史**重发**一遍。

参数在推理时不更新；状态只存在于上下文里。

训练：预训练与后训练

阶段 01 · PRETRAINING

预训练

大规模文本、代码等语料
学习语言、知识与模式

产出：基础模型 · base model

→

阶段 02 · POST-TRAINING

后训练

SFT · 监督微调

从示范学习

学习回答方式与任务行为。

RL · 本节重点

通过奖励学习

尝试、评价、改进策略。

> 重点讲 **RL**：奖励如何推动**问题求解能力**提升。

RL：行动、结果与反馈

在环境里行动，根据结果反馈，逐步改进动作选择。

观察环境 → 采取动作 · Action → 得到结果 → 反馈 / 奖励 → 改进策略 ↺

环境 · ENVIRONMENT

产生结果

动作改变环境，返回新的观察。

反馈 · FEEDBACK

评价结果

用奖励表达进展与成败。

策略 · POLICY

改进动作选择

根据经验调整，追求长期回报。

RL 根据**结果反馈**，学习**怎样选择动作**。

参考：Sutton & Barto, Reinforcement Learning: An Introduction。

游戏，让强化学习的能力被看见

2015

Atari · DQN

像素 → 动作

奖励：游戏分数

2016

AlphaGo

自我对弈 + 搜索

奖励：赢棋

2019

OpenAI Five

Dota 2 团队协作

奖励：比赛结果

2019

AlphaStar

StarCraft II 策略

奖励：比赛结果

- 规则明确
- 结果可衡量
- 可反复尝试

> 游戏引擎同时提供**环境与反馈**。

来源：Mnih et al., 2015 · Silver et al., 2016 · OpenAI Five, 2019 · Vinyals et al., 2019。

一个分数，怎样教会一种策略?

打砖块：移动挡板，接住球，清除砖块。

尝试

不同动作

探索时机与方向。

观察

结果得分

漏球，或清除更多砖块。

学习

更好的策略

学会具有延迟收益的动作。

反复尝试 + 结果反馈 → 学习取得更高回报的策略。

有用的环境，让智能体**尝试、评价、再尝试**。

例子：Mnih et al., Nature (2015), Breakout 演示。

GRPO 的目标函数

DeepSeekMath：组内相对优势，加上裁剪与 KL 约束。

$$J_{\mathrm{GRPO}}\left(\theta\right)=E_{q\sim P(Q),\left\{o_i\right\}_{i=1}^{G}\sim\pi_{\theta_{\mathrm{old}}}}\left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{\left|o_i\right|}\sum_{t=1}^{\left|o_i\right|}L_{i,t}\right]$$

逐 TOKEN 项

$$L_{i,t}=\min\left[\rho_{i,t}\left(\theta\right)\hat{A}_{i,t},\mathrm{clip}\left(\rho_{i,t}\left(\theta\right),1-\varepsilon,1+\varepsilon\right)\hat{A}_{i,t}\right]-\beta D_{\mathrm{KL}}\left[\pi_\theta\middle\|\pi_{\mathrm{ref}}\right]$$

概率比

$$\rho_{i,t}\left(\theta\right)=\frac{\pi_\theta\left(o_{i,t}\mid q,o_{i,<t}\right)}{\pi_{\theta_{\mathrm{old}}}\left(o_{i,t}\mid q,o_{i,<t}\right)}$$

组内优势

$$\hat{A}_{i,t}=\frac{r_i-\mathrm{mean}\left(r_1,\ \ldots,r_G\right)}{\mathrm{std}\left(r_1,\ \ldots,r_G\right)}$$

提高高优势回答的概率，同时用 clip 与 KL 限制更新幅度。

来源：Shao et al., *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models*, arXiv:2402.03300, 2024 · 公式按 GRPO 目标整理；实现细节可能不同。

GRPO：同一道题，比较一组回答

不用 critic，用组内相对表现构造优势。

同一问题 → 采样一组回答 → 逐个评分 → 组内相对优势 ↺

同一问题 · G = 4 个回答

奖励 / 组内优势 / 更新方向

A · r = 0.20	−1.5 ↓
B · r = 0.90	**+1.1 ↑**
C · r = 0.50	−0.4 ↓
D · r = 0.80	**+0.7 ↑**

均值 0.60，标准差 0.27；数值为示意。

组内竞争

同一题的回答互相比较，不跨题比较绝对分数。

向更好的结果学习

正优势回答提高概率；负优势回答降低概率。

不需要 CRITIC

组均值就是基线，省掉单独的价值模型。

同一道题里，让**更好的回答胜出**。

REFLEX：把搜索引擎放进 RL 闭环

电商 Query 扩展：生成 → 检索 → 用结果质量给奖励。

Query + top-k 商品 → LLM 生成扩展 → 搜索引擎 → 结果并集 → 奖励 ↻

奖励 · 无需人工标注

相关性 Rel(Y)/Rel(q) + 覆盖 Ret(Y)/Ret(q)；格式不合规记 0。

训练

SFT 预热 1 万条 → GRPO 10 万条真实 Query；8×H200。

正增益 QUERY 占比 · %

设置	ΔCOV	ΔREL
仅 SFT	12.8	8.9
仅 GRPO	57.5	25.2
预热 + GRPO	72.8	32.4

10,000 queries · 5 samples · LLaMA 3.2-3B

> GRPO 把“检索结果更好”变成**可优化的奖励**。

Zhang, Liu, et al., REFLEX, WSDM 2026 · 结果为该数据集与骨干上的消融。

数学与代码，共享一类反馈优势

01 · 外部环境封闭

公理系统 · 运行环境

数学靠定义、公理与推理规则；代码靠语言规范与运行时。系统自足，不需要外部信息。

02 · 错误可追溯

定位到具体步骤

证明逐步检查，找到失效步骤；测试、异常栈与行号定位代码错误。

03 · 数据干净

可解析、可筛选

形式化对象与代码结构清晰，便于批量生成、过滤与验证。

两条路线：学习更好的策略 + 用算力生成、验证更多候选。

封闭环境 + 可定位错误 + 干净数据 → **反馈便宜，可大规模试错。**

“封闭”是相对任务而言：代码仍依赖输入与库，数学建模仍需形式化；数据也仍需筛选。

代码与数学，训练共享的推理技能

编程训练

- 追踪变量与状态
- 遵守类型与约束
- 分解问题，定位错误

数学训练

- 明确写出假设
- 保持符号与逻辑一致
- 建立并检查多步论证

已有迁移证据： 数学 RL → 代码表现；代码训练 → 状态追踪。

共享技能支持**能力迁移**，数据配比影响实际收益。

来源：Chen et al., AceReason (2025) · Kim et al. (2024) · Shao et al., DeepSeekMath (2024)。

编程，为 Agent 提供了行动场所

读文件 → 修改 → 跑测试 → 观察与修订 ↺

工作目录

文件与规范定义任务。

真实工具

修改可以立即执行。

结果反馈

失败指导下一次修改。

例子：Claude Code、Codex、OpenCode，以及其他 coding agents。

> 代码在同一工作目录连接**生成、行动与反馈**。

更早的生成与筛选案例：Li et al., AlphaCode, Science (2022)。

四类任务，不同的反馈条件

编程、数学、产品设计、交易：目标、环境与反馈怎样配合?

01 · 编程

可执行的检查

- 目标：满足指定功能与规范
- 反馈：编译、测试、报错
- 试验：本地反复运行与修订

02 · 数学

明确的假设与目标

- 目标：在给定假设下求解或证明
- 反馈：答案检查、证明检查、反例
- 试验：反复生成与验证候选

03 · 产品设计

长周期的现实反馈

- 目标：用户价值与差异化
- 反馈：更慢、多元，需现实证据
- 判断：独特路线可能来自非共识

04 · 交易

清晰奖励，变化的环境

- 目标：盈亏、风险与成本
- 环境：信息不完备，市场持续变化
- 验证：回测与未来收益有距离

> 可检查的目标，让更多算力能用于**系统搜索与候选筛选**。

对有限、有解且可可靠判定的任务，穷举能找到满足规范的解；计算成本仍需考虑。

正确是底线，优秀是追求

	代码	数学
可检查结构	语法、类型、测试、规范	**定义、假设、证明步骤**
正确性	满足指定行为	**命题从假设推出**
优秀程度	清晰、可维护、高效	**优雅、一般、有用**

结构化产物让检查更容易；数据质量仍需要筛选与整理。

> 两个领域都追求**好作品**，超越一次通过或失败。

总结：环境、反馈与参数更新

良好定义的问题，让模型可以在环境中反复预测、接受检查并迭代。

问题与环境 → next-token 预测 → 检查与追溯 → 定义 loss → 反向传播 ↺

01 · 环境

规则与目标明确

模型可以执行、尝试并产生输出。

02 · 反馈

评判与追溯

检查结果，定位出错步骤。

03 · 更新

loss 驱动迭代

反向传播，更新参数，提升能力。

> 环境提供反馈，**loss 把它变成参数更新**。

模型会预测，但还有两个短板

RL 之后，能力仍在模型内部；记忆、计算与验证要由系统补上。

短板 01 · 没有记忆

每次调用独立

“对话”是客户端把历史重发一遍；没有前文，“它”就失去指代。

短板 02 · 运算不可靠

按分布生成，不执行程序

算术看起来合理，也可能算错；精确计算要交给程序。

两个补足方向：把状态与执行放到模型外部。

CONTEXT 管理

把历史、资料与约束带回窗口；记忆与检索是外部机制。

HARNESS · 工具调用

把精确计算与行动交给程序和环境，并检查结果。

模型负责预测；**记忆、计算与验证**由系统补上。

从会说话到会干活

两条补足线：Context 管理状态与历史；Harness 管理计算与执行。

会说话 · 2023 · CONTEXT

ChatGPT

指令微调 + RLHF
prompt / system

会思考 · COT · CONTEXT

CoT

把中间步骤写进上下文
context engineering · pruning

会干活 · AGENT · HARNESS

Agent

tool calling · loop
外部状态与验证

A · 状态线：Prompt → Context → Memory　B · 执行线：Tools → Agent → Harness

两条线，补两个短板。

2023：从补全到对话

user 用一句话解释希尔区。

assistant 给定能量下，天体可能出现的空间区域。

指令微调

学会对话格式：该回答，而不是续写

RLHF

学会人觉得好的回答：有用、无害

但仍然不会

不会用工具，不会记事，不会验证

> 会说话，是训练出来的。

Prompt：用内容改变分布

SYSTEM · 角色
你是天体力学助教。

CONSTRAINTS · 约束
先写公式，再代入，最后给数值。

USER · 任务
周期 2 年，半长轴变为 4 倍，周期变成多少?

FEW-SHOT · 例子
a 变为 9 倍 → T 变为 27 倍
a 变为 16 倍 → T 变为 64 倍



$$p_\theta(y \mid x) \rightarrow p_\theta(y \mid x, c, e)$$

p(16 | x) = 0.18
p(16 | x, c, e) = 0.71

> 加内容，就是在改**条件概率**。

Context window 的膨胀



贵

按 token 计费；长上下文预填更慢

慢

prefill 与注意力成本上升

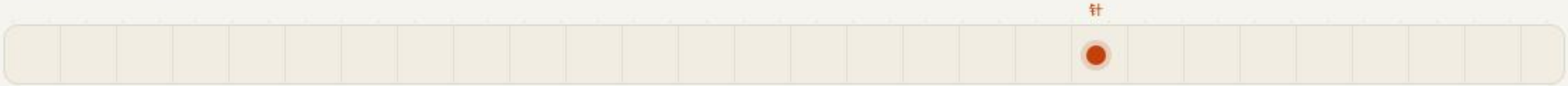
稀释

中间信息被忽略；信号被噪声淹没

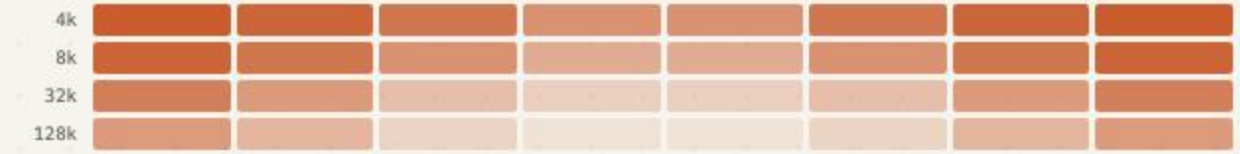
> 信息密度比窗口长度更重要。

Context 是有限的

大海捞针：在长文里藏一句话



检索准确率（示意）



横轴：针的位置 0 → 100%

怎么做

藏一句话，换长度、换位置，问它是什么

已观察到的失效模式

长度与位置可能影响检索；具体表现随模型和任务变化

含义

窗口大 ≠ 找得到

示意图，非实测数据 · 参见 Liu et al., Lost in the Middle (2023)。

> 窗口容量与**有效利用上下文**，是不同的指标。

needle in a haystack

Context engineering：把窗口当工作台

窗口里装什么

- system：规则
- 历史：前几轮
- 检索：相关文档
- 工具：返回结果
- 当前：这次的问题

太长

检索，只放相关的

太散

结构化，分块标注

中间丢失

关键信息放首尾

超窗

压缩、摘要、子 agent

> 上下文越相关越好。

CoT：把思考写出来

范围：言语与逻辑推理；算术是另一件事。

问题

周期 2 年，半长轴变为 4 倍，周期变成多少?

直接答

8 年 ✗ —— 一个没有计划的数字

COT · 计划

$T \propto a^{3/2}$；$T = 2 \times 4^{3/2}$

计划 ✓ · 算术下一步

工具 · 计算

```
run_python("2 * 4**1.5") → 16.0 ✓
```

> CoT 负责写计划；程序负责算数。

COT 做什么

把文字题拆成计划：识别、回忆、设关系

剩下什么

算术：另一件适合工具的事

本质

显式步骤；精确数字是另一项能力

CoT pruning：只写值得写的思考

一条长思维链

读题 → 设变量 → 展开 → ~~重复展开~~ → 分类讨论 → ~~无关分支~~ → 回到主线 → 代入 → 验算 → 答案

剪枝

去掉重复、无关的分支

蒸馏

用长 CoT 教短 CoT

摘要

长链压成短链

预算

限制思考 token，早停

> 思考也有预算。

Memory：信号与噪声

窗口里的内容

信号

任务 · 约束 · 决定 · 已验证事实

噪声

闲聊 · 重复 · 过期 · 原始日志

写入

只存决定与事实

检索

按当前任务取

压缩

摘要、合并

遗忘

过期即删

工作记忆

当前窗口；只放这次要用的

情节记忆

历史摘要；决定与未决问题

语义记忆

外部库 + 检索；按需取

程序记忆

system + tools；规则与能力

> 记忆的关键是取得准。

Tool calling：算不准，就交给程序

```
工具：run_python(code: str) → str

call   run_python("print(2 * 4**1.5)")
obs    16.0

// T = 2 × 4**1.5 = 16
```

01 定义

工具签名：名字、参数、返回

02 调用

模型输出一段代码

03 执行

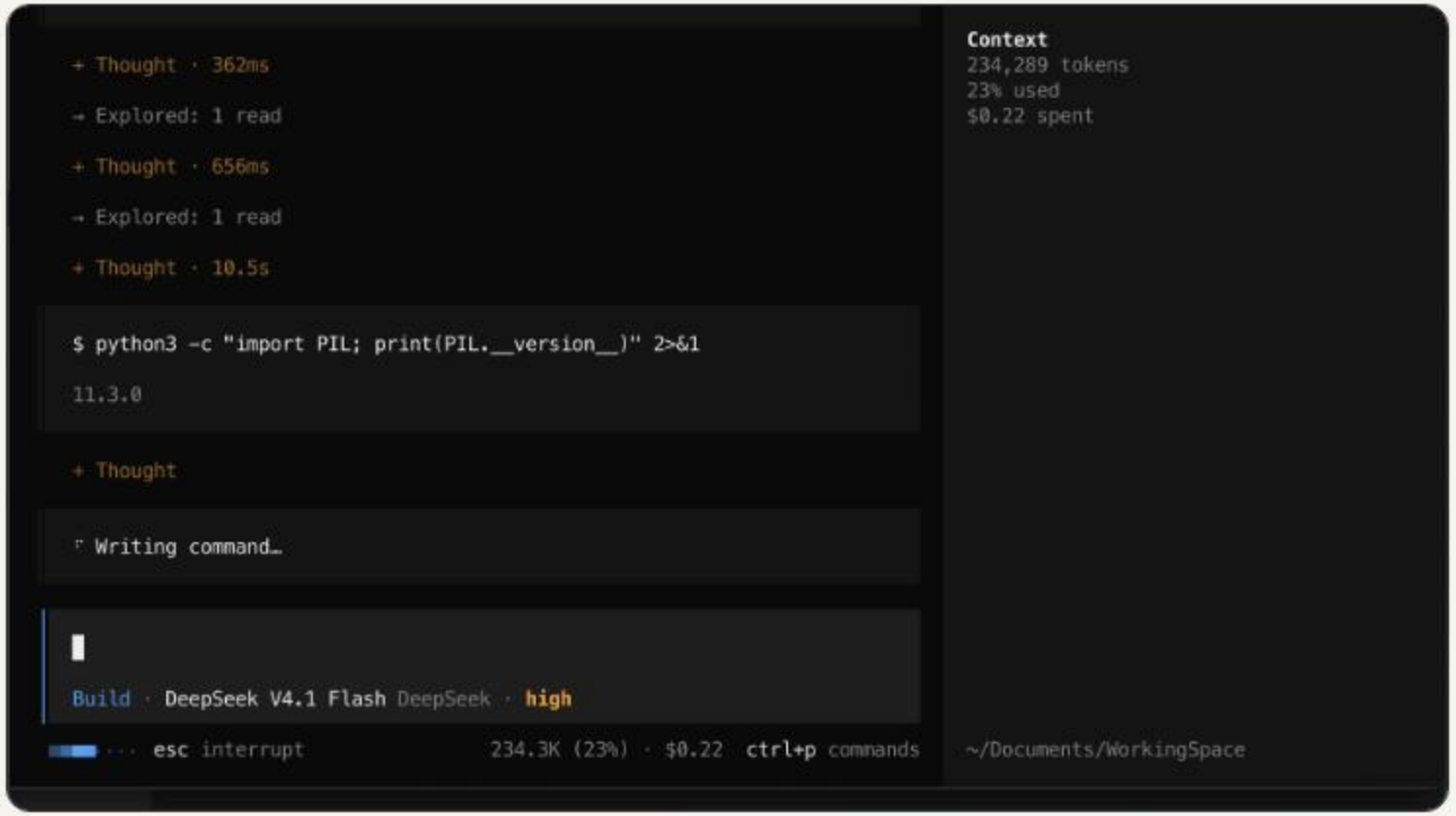
沙箱里真的跑

04 观察

结果回到上下文，继续

> 模型负责提议，程序负责算对。

Agent：模型被放进一个循环



01
循环
Thought → 工具调用 → 观察，重复

02
上下文
234K tokens · 23% used · $0.22

03
客户端
模型 + 工具 + 会话 + 账单

04
模型
DeepSeek V4.1 Flash · high

> 模型负责提议，客户端负责把提议变成**行动**。

代码产出激增，人工审查来不及了

Claude Code 团队的卡点：代码增长得比人的注意力更快。

产出压力 · ANTHROPIC MONOREPO

2,500 → 30,000+

每日落地变更 · 2026 年 5 月 → 9 月

Claude Code 使用另一个独立仓库。

生产端

Agent 更快地产出更多代码。

人的卡点

人的审查时间与注意力有限。

结果

逐行人工审查跟不上产出。

目标：减少**审查 + 修复的总人时**，同时以回滚率与事故率把关。

下一步收益，来自**重新组织审查流程**。

来源：Osmani、Lunt、Wang · 2026-10-15 · How the Claude Code team does code review · 团队自述实践。

用系统接住常规审查

检查贯穿工作流程，人的注意力留给关键判断。

01 · 提交之前

先产出检查证据

- 测试、构建、检查界面
- 新上下文的 Agent 审查
- 提交审查前先修一轮

02 · 审查之中

分层发现问题

- AI 审查，复核发现
- 安全检查与自动测试
- 每项发现都要处理

03 · 批准与发布

控制剩余风险

- 按风险安排人工批准
- 合并检查，分阶段发布
- 快速发现，快速回滚

共享上下文： CLAUDE.md 存项目约定；REVIEW.md 存局部失败标准与证据要求。

常规检查持续运行，人聚焦**判断与风险**。

来源：Osmani、Lunt、Wang · 2026-10-15 · How the Claude Code team does code review · 团队自述实践。

人的注意力，按风险分配

团队的审批策略，以及明确的责任归属。

变更风险	例子	人工审批
低	文档、文案、仅内部使用的功能	**可免人工审批***
中	常规产品变更	**1 人***
高	最高风险代码、指定路径	**2 人**

* 较轻流程仅对可信作者开放，且须通过必要检查；敏感路径可能要求额外审批。

方向

人判断这件事是否值得做。

证据

批准针对某个明确的提交。

责任

指挥 Agent 的人对变更负责。

> 需要人的审批位由**人**担任，每次变更有**明确负责人**。

来源：Osmani、Lunt、Wang · 2026-10-15 · How the Claude Code team does code review · 团队自述实践。

让审查持续改进，也能停下来

失败进入评测集，审查循环设置升级处理条件。

事故 / 回滚 → 工程师确认的案例 → 每日评测 → 改进审查器 ↻

同时衡量质量与成本

- 发现了多少真实错误
- 对干净变更产生多少误报
- 经过几轮才能通过审查

给循环设置边界

- 三轮后重新审视方案
- 后续聚焦新增变更
- 按错误类别修复，避免重复

原文记录的失败模式：一次变更在夜间经历了 50 多轮审查与修复。

> 把失败沉淀为**可复用的检查**，让审查经验持续**复利**。

来源：Osmani、Lunt、Wang · 2026-10-15 · How the Claude Code team does code review · 团队自述实践。

把模型放回系统里



内圈：一次调用 · 中圈：跨调用 · 外圈：工程边界

01	**LLM**	概率生成器：输入 token，输出下一个 token 的分布
02	**Prompt**	这次的条件：system / user / few-shot
03	**Context**	这次能看到的全部 token；窗口有限
04	**Memory**	窗口外的状态：写入 / 检索 / 压缩 / 遗忘
05	**Agent loop**	观察 → 行动 → 观察；状态放到外部
06	**Harness**	工具、沙箱、权限、验证、日志

> 模型生成**提议**；系统管理上下文与状态，执行工具并**检查结果**。

加速：从基本应用到金牌

2023 · 之前

基本应用

- 解释概念、查文献、写代码
- 做简单计算，经常算错
- 复杂证明仍要人
- 角色：助手

→ 2024–26

2025–26 · 之后

研究级

- IMO 2025 金牌级 35/42
- Lean 形式证明可验
- OpenAI 2026：每项结果约 3 Pro 小时算力*
- 角色：候选生成器

2024.07 **28/42** 银牌 → 2025.07 **35/42** 金牌级　12 个月，+7 分

* 算力等价值，非实际耗时 · OpenAI, 2026-10-06 · 公司报告。

> 这是一条阶梯：**三个台阶**。

时间线：三个台阶

2023–24

工具：搜索 + 验证

2023.12
FunSearch
LLM + evaluator + 进化搜索；cap set 新构造

2024.01
AlphaGeometry
神经符号；IMO 几何 25/30

2024.07
AlphaProof
Lean + RL；IMO 2024 银牌 28/42（+AG2）

2024–25

推理：RL + 测试时计算

2024.09
OpenAI o1
RL 学思考；test-time compute

2025.01
DeepSeek-R1
RLVR；推理行为涌现；开放权重

2025.05
AlphaEvolve
进化编码 agent；4×4 复矩阵乘法 48 次

2025–26

规模：从 benchmark 到开放问题

2025.07
IMO 2025
金牌级 35/42；自然语言、无工具

2025
形式化金牌（声称）
Aristotle 等；Lean 可验

2026.05–10
OpenAI
单位距离反例；NS / 黎曼相关（声称）；372 个问题（报告）

> 每一次跃迁，都是**生成器 + 验证器 + 搜索**。

他们怎么说的

> 公开材料覆盖了方法的一部分；完整的训练配方仍未公开。注意一种不对称：有些结果验证容易，证明很难。下面是论文、博客和赛后声明里的**公开表述**（节选 / 意译）。

时间	系统 / 工作	公开表述	谈法
2023.12	**FunSearch**	"pairing a pre-trained LLM ... with an automated evaluator"	搜索
2024.01	**AlphaGeometry**	"100 million synthetic theorems and proofs"	合成数据
2024.07	**AlphaProof**	autoformalization → Lean; RL + test-time RL	形式化
2024.09	**o1**	"spend more time thinking through problems"	推理
2025.01	**R1**	"directly apply RL ... rule-based rewards"	RLVR
2025.05	**AlphaEvolve**	"evolutionary coding agent"	进化搜索
2025.07	**IMO 2025**	"natural language ... no tools"	端到端
2026.09–10	**OpenAI**	"NS solution; 372 problems; 722 papers"	开放问题

> 关键词在变，底层动作没变：**提议、验证、再搜索**。

共同的模式：生成 → 验证 → 搜索

生成

LLM

候选证明、程序、构造

→

验证

Lean / 测试 / 数值

给出对错、分数、反例

→

搜索 / 学习

best-of-N · 树 · 进化 · RL

把信号变成下一步策略

↺ 回到生成器：下一轮更强

没有验证器

只能靠人看；无法规模化

没有搜索

一次生成；好答案靠运气

两者都有

概率被筛成正确，再写回模型

> 模型提供概率，**验证器提供梯度**。

一些猜测：大结果可能是怎么来的

以下机制假设，来自公开论文、博客和 benchmark 的交叉印证，标注为**猜测**。

01 数据

自动形式化 + 合成：非形式题转 Lean；拒绝采样留下可验证的。

02 训练

RLVR + 长 CoT：答案可验证，奖励才能规模化。

03 推理

Test-time search：best-of-N、树搜索、进化。

04 蒸馏

把搜索写回模型：下一次不用搜那么久。

05 工程

Harness：并行、缓存、评测、回放。

06 赌注

验证：谁有更硬的 verifier，谁就有更快的飞轮。

OpenAI 报告：每项结果约 3 小时 ChatGPT Pro 等价算力 · 2026-10-06。

已知算力预算，**系统设计仍有多种可能**。

还没解决的：从解题到做数学

已经发生

竞赛题

IMO 金牌级；自然语言、限时

形式证明

Lean 可验；不再依赖人眼

构造搜索

cap set、矩阵乘法、调度

开放问题

OpenAI 报告 372 个；筛选仍靠人

还没

选题

什么问题值得做

定义

新概念、新结构

长程

几个月的研究计划

品味

美、重要、可生长

> 解题可以搜索；**做数学需要方向**。

电机与流水线

Eric Schmidt 讲过一个故事；AI 正在经历同一件事。

电机刚被发明的时候，工厂主的做法很直觉：把中央蒸汽机拆掉，换一台电机，其他一切不变。效率提高了大约 30%。

过了几十年，有人意识到电机可以做得很小。每台机器可以有自己的电机。工厂可以围绕生产流程本身来设计；动力源变成每台机器自己的事。

流水线诞生了。效率提高了几十倍。

30%

单机模式 · Prompting Harder
换窗口即归零 · 无法积累

10×+

流水线 · Context Engineering
沉淀规则与模板 · 持续复利

30% 和几十倍的差距，是**可积累的工作方式**。

> 模型是租来的，上下文与系统才是自己的：别只在旧步骤里「问一下 AI」，把规则、工具与检验沉淀为长期资产。

收益在**布局与复利**。

算法工程师的困境

这个问题在一年前出现；数学正在经历同一件事。

“如果模型会写代码，我的工作是什么？”

之前

写算法

- 实现、调试、优化
- 对代码负责
- 价值 = 交付代码

→ 约一年

现在

指挥搜索

- 定义问题
- 设计 harness
- 验证、对系统负责

> 工作从「产出答案」转向**「指挥搜索」**。

从网页对话框到工作目录

对我的长任务，工作目录让规则与结果可以持续复用。

模式 A · 对话

网页对话

讨论与探索

提问、解释、上传资料

工具能力取决于配置

网页也可以提供工具与记忆

模式 B · 编辑器

IDE 编辑器

紧贴代码进行编辑

文件、诊断、仓库上下文

辅助编辑或运行 Agent

能力取决于工具与权限

模式 C · 工具循环

工作目录 Agent

工具集成与子智能体

Python 辛积分 · 摄动展开 · 隔离审查

自主闭环与版本可控

假说 → 实验 → 守恒校验 → git 归档

- OpenCode
- Claude Code
- Codex CLI
- Cursor
- Markdown SOP 协议

> 文件即接口；在 **本地工作目录** 沉淀可移植的科研资产。

文件与检查可跨工具复用；模型或环境变化后，重新运行评测。

两次尝试

同样的目标，不同的治理方式：从放任到做 Coach。

OPENCLAW · 早期尝试

误当计算器 · 完全放任

- 试图让 agent 端到端全包写完论文。
- 早期模型弱、幻觉多，且无任何外部检验回路。
- 人类彻底退场，期待自动化奇迹。

我的结果：没有完成论文；引导与检查不足。

CODEX · GPT-5.3–5.5 · ISTART

做 Coach · 建立验证回路

- 博后工作延伸；没时间手写，由我定义清晰目标（WHAT）。
- 不做逐句修改的 Fixer，做指出破绽的 Coach。
- 纠正荒谬推导：$Ax = b,\ b \neq 0 \to x = 0$ ✕
- 拒稿 3 次后，用 GPT-6.1 Sol 扫出深层隐患并修正。

成效：验证不对称性——Eval 比生成更容易。

> 不做救火的 Fixer，做定标准与反例的 **Coach**。

低垂的果实与长期复利

生成论文只是起点；可复利的资产是评估器与科研品味。

01 · 人类作为 COACH

定目标与反例

明确定义 WHAT，放手让 AI 探索 HOW；给出逻辑反例，拒绝人工逐行代修。

02 · 系统化的 EVAL

动力学检验

能量误差行为、辛性残差、步长收敛测试，提供互补的检查证据。

03 · 可复利资产

品味与评估集

单篇论文是易贬值的果实；持久复利的是知识库、代码 SOP 与形式化评测器。

当起草变得更便宜，我更愿意投入**选题、可复用知识与评估系统**。这些资产可以跨项目积累，形成长期复利。

> 下一个核心问题：哪个问题**值得机器与人类一起做**?

年轻人的机会

天体力学正在变成数据领域；稀缺的是找到信号的能力。



数据

Gaia、卫星跟踪、巡天、N 体模拟。

数据比手多。



手艺

去噪、建模、验证；agent 跑流水线。

新的杠杆。



信号

共振、异常、新轨道。

问题还是老的。

数据比手多，工具是新的，问题是老的。**这就是年轻人的机会。**

> 瓶颈在**信号**。

从下一个 token 到下一个定理

token → 概率 → 候选 → 验证 → 定理

概率负责广度

验证负责正确

人负责方向

> 下一个定理，仍然从**下一个 token** 开始。